

Тема: Наследование

Например: На основе предметной области "Служащий, персона, рабочий, инженер" можно спроектировать иерархию классов, используя принципы абстракции¹ и наследования.

Иерархия классов

Person (Персона) — абстрактный базовый класс:

Абстракция: Это самый общий класс, представляющий любого человека. Он содержит общие свойства и поведение, присущие всем людям, независимо от их занятости.

Свойства: `name` (имя), `age` (возраст).

Абстрактные методы: Определим абстрактный метод, который должен быть реализован в каждом подклассе, например, `getRole()` для получения информации о роли человека.

Обычные методы: Метод `toString()` для вывода базовой информации.

Employee (Служащий) — абстрактный класс-наследник:

Наследование: Наследуется от класса **Person**.

Абстракция: Этот класс представляет собой более конкретную сущность — человека, который является сотрудником. Он добавляет общие для всех сотрудников свойства и методы.

Свойства: `employeeId` (идентификатор сотрудника), `salary` (зарплата).

Абстрактные методы: `calculateSalary()` (расчёт зарплаты, который будет разным для разных типов сотрудников).

Обычные методы: `getHireDate()` (получение даты найма).

Worker (Рабочий) — конкретный класс-наследник:

Наследование: Наследуется от класса **Employee**.

Конкретизация: Это конкретный тип служащего.

Свойства: `hourlyRate` (часовая ставка), `hoursWorked` (отработанные часы).

Реализация абстрактного метода: Переопределяет метод `calculateSalary()`, чтобы вычислять зарплату на основе отработанных часов и часовой ставки.

Engineer (Инженер) — конкретный класс-наследник:

Наследование: Наследуется от класса **Employee**.

Конкретизация: Ещё один конкретный тип служащего.

Свойства: `department` (отдел), `bonus` (премия).

Реализация абстрактного метода: Переопределяет метод `calculateSalary()`, чтобы вычислять зарплату с учётом фиксированной ставки и премии.

Принцип проектирования

Абстракция: Классы **Person** и **Employee** абстрагируют общие черты и поведение, которые присущи всем производным классам.

Наследование: **Worker** и **Engineer** наследуют свойства и методы от **Employee**, а тот, в свою очередь, от **Person**. Это позволяет избежать дублирования кода.

Полиморфизм: Метод `calculateSalary()` будет вести себя по-разному в зависимости от того, какой конкретный класс его вызывает (**Worker** или **Engineer**).

Инкапсуляция: Каждое свойство класса инкапсулируется внутри него, доступ к нему осуществляется через методы.

¹ Абстракция -это принцип ООП, согласно которому при проектировании классов и создании объектов необходимо выделять только главные свойства сущности, и отбрасывать второстепенные.

Реализовать консольное приложение
для предметных областей указанных в индивидуальном задании
удовлетворяющее следующим требованиям:

Цель: Реализовать полиморфную систему на основе абстрактного класса, используя механизмы наследования и переопределения методов.

1. Архитектурная задача

Вам необходимо создать **абстрактный базовый класс** (согласно вашему варианту) и **три конкретных класса-наследника**.

- **Базовый класс:** Содержит общие поля (состояние), конструктор и абстрактный метод для основного действия.
- **Классы-наследники:** Реализуют абстрактный метод родителя и добавляют по одному **уникальному методу**, расширяющему логику конкретного объекта.

2. Требования к реализации (Логика и Полиморфизм)

- **Полиморфное поведение:** Реализация общего метода в каждом наследнике должна быть уникальной (разные алгоритмы или сообщения), но соответствовать смыслу действия.
- **Управление состоянием:** Выполнение методов должно зависеть от полей класса. Метод может быть выполнен только если объект находится в «рабочем» состоянии (например, прибор включен, животное не спит, на счету достаточно средств).
- **Использование `super`:** В конструкторах наследников обязательно вызывать конструктор базового класса для инициализации общих полей.

3. Стратегия обработки ошибок (L1–L3)

Применяйте изученную ранее иерархию:

- **L1 (Саморегуляция):** Если метод вызван в неподходящем состоянии, объект должен попытаться исправить это сам (например, автоматически включиться перед работой).
- **L2 (UI):** Если исправить ситуацию невозможно, программа должна через меню предложить пользователю выполнить необходимое действие (например, «Сначала заправьте бак»).
- **L3 (Exception):** Выбрасывайте исключение только при критическом нарушении логики, которое невозможно решить на уровне кода или пользователя.

4. Интерфейс управления (Меню)

В программе должен быть реализован список (массив или `ArrayList`) типа базового класса, содержащий объекты разных наследников. Меню должно позволять:

1. **Создать объект:** Выбрать тип наследника и задать его параметры.
2. **Групповое действие:** Одной командой заставить все созданные объекты выполнить свой основной (полиморфный) метод.
3. **Специфическое действие:** Вызвать уникальный метод конкретного объекта.

Индивидуальные задания

1. Бытовая техника (Технический профиль)

- **Базовый класс:** `Appliance` (бренд, мощность, состояние: включен/выключен).
- **Методы:** `turnOn()`, `turnOff()`, абстрактный `work()`.
- **Наследники:**
 - o `WashingMachine`: метод `startLaundry()` (уникальный: выбор режима стирки через enum).
 - o `Refrigerator`: метод `setTemperature(int t)` (уникальный: проверка допустимого диапазона).
 - o `VacuumCleaner`: метод `changeBag()` (уникальный: сброс счетчика заполненности).
- **Логика:** Метод `work()` не должен выполняться, если прибор выключен (`isPoweredOff`).

2. Банковская система (Финансовый профиль)

- **Базовый класс:** `BankAccount` (номер счета, баланс, владелец).
- **Методы:** `deposit(amount)`, абстрактный `withdraw(amount)`.
- **Наследники:**
 - `SavingsAccount`: метод `applyInterest()` (начисление годовых процентов).
 - `CreditAccount`: метод `checkDebt()` (проверка задолженности по лимиту).
 - `BusinessAccount`: метод `sendInvoice(amount)` (выставление счета со специфической комиссией).
- **Логика:** `withdraw` должен проверять наличие средств. Если это кредитный счет — учитывать кредитный лимит (L1-логика).

3. Зоопарк (Биологический профиль)

- **Базовый класс:** `Animal` (кличка, возраст, уровень голода).
- **Методы:** `eat()`, абстрактный `makeSound()`.
- **Наследники:**
 - `Lion`: метод `hunt()` (уникальный: повышает уровень сытости, но тратит энергию).
 - `Elephant`: метод `sprayWater()` (уникальный: охлаждает себя, меняя поле «температура»).
 - `Monkey`: метод `climbTree()` (уникальный: проверка высоты дерева).
- **Логика:** Если уровень голода критический, `makeSound()` должен выводить предупреждение о необходимости кормления.

4. Музыкальный магазин (Творческий профиль)

- **Базовый класс:** `MusicalInstrument` (материал, цена, строй).
- **Методы:** `tune()`, абстрактный `play()`.
- **Наследники:**
 - `Piano`: метод `pressPedal()` (уникальный: меняет длительность звучания).
 - `Guitar`: метод `replaceString(int number)` (уникальный: влияет на качество строя).
 - `Drum`: метод `tightenMembrane()` (уникальный: меняет высоту тона удара).
- **Логика:** Если инструмент не настроен, метод `play()` должен выдавать «фальшивый» звук.

5. Электронные документы (Бизнес-профиль)

- **Базовый класс:** `Document` (ID, автор, статус: черновик/подписан).
- **Методы:** `edit()`, абстрактный `sign()`.
- **Наследники:**
 - `Invoice`: метод `calculateTax()` (уникальный: расчет НДС в зависимости от суммы).
 - `Contract`: метод `extendValidity(int days)` (уникальный: продление срока действия).
 - `Report`: метод `attachChart()` (уникальный: добавление графических данных).
- **Логика:** Метод `edit()` должен блокироваться, если документ уже имеет статус «подписан».

6. Космическая станция (Научный профиль)

- **Базовый класс:** `SpaceObject` (название, масса, координаты).
- **Методы:** `updateCoordinates()`, абстрактный `getInfo()`.
- **Наследники:**
 - `Planet`: метод `calculateGravity()` (уникальный: расчет силы тяжести на поверхности).
 - `Star`: метод `emitFlare()` (уникальный: выброс энергии, меняющий светимость).
 - `Comet`: метод `getTailLength()` (уникальный: зависит от расстояния до центра координат).
- **Логика:** Если масса объекта становится отрицательной (ошибка ввода), объект должен скорректировать её до 1 (L1) или выдать ошибку (L3).

7. Спортивный инвентарь (Активный профиль)

- **Базовый класс:** `SportsEquipment` (состояние износа, вес, материал).
- **Методы:** `service()`, абстрактный `use()`.
- **Наследники:**
 - `Ball`: метод `inflate(double pressure)` (уникальный: изменение давления внутри).
 - `Racket`: метод `restring()` (уникальный: замена струн, сброс износа).
 - `Skates`: метод `sharpen()` (уникальный: заточка лезвий, влияет на «скольжение»).
- **Логика:** Если износ составляет 100%, метод `use()` запрещает использование предмета (L2/L3).

8. Магазин мебели (Дизайн-профиль)

- **Базовый класс:** `Furniture` (габариты, цена, цвет).
- **Методы:** `move()`, абстрактный `assemble()`.
- **Наследники:**
 - `Chair`: метод `adjustHeight(int h)` (уникальный: проверка пределов газлифта).
 - `Table`: метод `expand()` (уникальный: увеличение площади поверхности поля).
 - `Wardrobe`: метод `addShelf()` (уникальный: изменение количества секций).
- **Логика:** Метод `move()` не может быть выполнен, если мебель в статусе «разобрана».

9. Кухонные гаджеты (Кулинарный профиль)

- **Базовый класс:** `KitchenTool` (материал, уровень чистоты).
- **Методы:** `wash()`, абстрактный `performAction()`.
- **Наследники:**
 - `Knife`: метод `sharpen()` (уникальный: повышает эффективность резки).
 - `Blender`: метод `changeSpeed(int s)` (уникальный: выбор режима вращения через enum).
 - `Toaster`: метод `setTimer(int t)` (уникальный: установка времени прожарки).
- **Логика:** Если прибор «грязный», метод `performAction()` должен требовать вызова `wash()`.

10. Авиация (Транспортный профиль)

- **Базовый класс:** `Aircraft` (бортовой номер, макс. высота, текущая скорость).
- **Методы:** `takeOff()`, абстрактный `fly()`.
- **Наследники:**
 - `Airplane`: метод `deployChassis()` (уникальный: выпуск шасси, проверка скорости).
 - `Helicopter`: метод `hover()` (уникальный: зависание в одной точке).
 - `FighterJet`: метод `eject()` (уникальный: катапультирование, уничтожение объекта).
- **Логика:** `takeOff()` невозможен, если текущая скорость ниже взлетной (L1: автоматический разгон или L2: сообщение пилоту).

11. Садовая техника (Эко-профиль)

- **Базовый класс:** `GardenTool` (тип питания: бензин/АКБ, уровень ресурса).
- **Методы:** `refill()` (заправка/зарядка), абстрактный `operate()`.
- **Наследники:**
 - `LawnMower`: метод `setCuttingHeight(int h)` (проверка: нельзя менять высоту при работающих ножах).
 - `Trimmer`: метод `extendLine()` (выпуск лески, тратит ресурс «длина лески»).
 - `LeafBlower`: метод `switchMode()` (переключение: выдув/пылесос через enum).
- **Логика:** `operate()` не запускается при ресурсе < 5% (L1: переход в эконом-режим или L2: сигнал оператору).

12. Оргтехника (Офисный профиль)

- **Базовый класс:** `OfficeDevice` (сетевое имя, статус: готов/ошибка/печать).
- **Методы:** `connect()`, абстрактный `processTask()`.
- **Наследники:**
 - `Scanner`: метод `setResolution(int dpi)` (проверка лимитов сканирующей головки).
 - `Copier`: метод `setCopyCount(int n)` (проверка наличия бумаги в лотке).
 - `Fax`: метод `dialNumber(String num)` (проверка корректности формата номера).
- **Логика:** Если `status == ERROR`, выполнение `processTask()` блокируется до вызова метода `reboot()`.

13. Средства индивидуальной защиты (Профильный)

- **Базовый класс:** `ProtectiveGear` (материал, уровень прочности в %).
- **Методы:** `inspect()`, абстрактный `protect()`.
- **Наследники:**
 - `Helmet`: метод `toggleVisor()` (поднять/опустить визор, меняет поле «обзор»).
 - `GasMask`: метод `replaceFilter()` (сброс уровня загрязнения фильтра).
 - `Respirator`: метод `checkTightness()` (проверка герметичности прилегания).
- **Логика:** При прочности < 20% метод `protect()` выводит предупреждение о критической опасности (L2).

14. Световые приборы (Светотехника)

- **Базовый класс:** `LightingFixture` (яркость в люменах, цветовая температура).
- **Методы:** `toggle()`, абстрактный `illuminate()`.
- **Наследники:**
 - `SmartLamp`: метод `setColor(String hex)` (изменение цвета через код).
 - `StreetLight`: метод `setPhotosensorThreshold(int lux)` (порог срабатывания от датчика света).
 - `Searchlight`: метод `rotate(int angle)` (проверка: угол вращения в пределах 0-360).
- **Логика:** При перегреве (поле `temperature`) яркость должна автоматически снижаться на 50%.

15. Системы хранения данных (IT-профиль)

- **Базовый класс:** `StorageDrive` (общий объем, занятое место, файловая система).
- **Методы:** `format()`, абстрактный `writeData(int size)`.
- **Наследники:**
 - `HDD`: метод `defragment()` (оптимизация, повышает скорость чтения).
 - `SSD`: метод `getHealthStatus()` (проверка оставшегося ресурса циклов перезаписи).
 - `FlashDrive`: метод `toggleWriteProtection()` (физическая блокировка записи).
- **Логика:** Если `writeData` превышает свободное место, метод должен предложить очистку или отменить операцию (L1/L2).

16. Оптические приборы (Физический профиль)

- **Базовый класс:** `OpticalDevice` (кратность увеличения, диаметр линзы).
- **Методы:** `cleanLens()`, абстрактный `view()`.
- **Наследники:**
 - `Binoculars`: метод `adjustDistance(double d)` (подстройка межзрачкового расстояния).
 - `Telescope`: метод `setTargetCoords(double x, double y)` (наведение на координаты).
 - `Magnifier`: метод `toggleBacklight()` (включение встроенной подсветки).
- **Логика:** Если линза загрязнена (поле `cleanliness` < 30%), метод `view()` выдает размытое изображение.

17. Системы климат-контроля (Инженерный)

- **Базовый класс:** `ClimateControl` (целевая температура, потребляемая мощность).
- **Методы:** `setTargetTemp()`, абстрактный `regulate()`.
- **Наследники:**
 - `AirConditioner`: метод `dryAir()` (режим осушения, меняет поле «влажность»).
 - `Humidifier`: метод `refillWater()` (заполнение бака водой).
 - `Ionizer`: метод `setIonLevel(int level)` (установка концентрации ионов).
- **Логика:** Если в увлажнителе закончилась вода, `regulate()` переходит в режим ожидания (L1).

18. Медицинские приборы (Медицинский профиль)

- **Базовый класс:** `MedicalDevice` (номер поверки, дата калибровки).
- **Методы:** `calibrate()`, абстрактный `takeMeasurement()`.
- **Наследники:**
 - `Thermometer`: метод `setUnit(enum C/F)` (переключение шкалы Цельсия/Фаренгейта).
 - `Tonometer`: метод `inflateCuff()` (накачка манжеты, проверка на перекачку).
 - `Glucometer`: метод `insertTestStrip()` (установка тест-полоски).
- **Логика:** Если дата последней калибровки превышает год, `takeMeasurement()` выдает ошибку точности (L3).

19. Инструменты для рисования (Арт-профиль)

- **Базовый класс:** `DrawingTool` (цвет, толщина линии).
- **Методы:** `setColor()`, абстрактный `draw()`.
- **Наследники:**
 - `Brush`: метод `dipInWater()` (увлажнение, влияет на прозрачность мазка).
 - `Pencil`: метод `sharpen()` (восстановление остроты, тратит длину карандаша).
 - `Airbrush`: метод `setPressure(double p)` (настройка давления распыления).
- **Логика:** Если карандаш сточен (длина = 0), метод `draw()` невозможен без `sharpen()` (L2).

20. Снаряжение для туризма (Активный отдых)

- **Базовый класс:** `CampingGear` (вес, вместимость в литрах/чел).
- **Методы:** `pack()`, абстрактный `setup()`.
- **Наследники:**
 - `Tent`: метод `stretchCanvas()` (натяжка тента, проверка устойчивости к ветру).
 - `Backpack`: метод `adjustStraps()` (подгонка лямок под рост).
 - `SleepingBag`: метод `zipUp()` (застегивание, изменение термоизоляции).
- **Логика:** Нельзя «упаковать» (метод `pack()`) палатку, пока она не «собрана» (L1/L2).

21. Системы безопасности (Охранный профиль)

- **Базовый класс:** `SecurityDevice` (статус: активен/ожидание, зона покрытия).
- **Методы:** `activate()`, абстрактный `alert()`.
- **Наследники:**
 - `MotionSensor`: метод `setSensitivity(int s)` (настройка порога срабатывания).
 - `SurveillanceCamera`: метод `rotate(int angle)` (проверка: поворот в пределах видимости).
 - `AlarmSiren`: метод `setVolume(int db)` (установка громкости оповещения).
- **Логика:** Метод `alert()` срабатывает только если `status == ACTIVE`. При низком заряде батареи датчик переходит в режим энергосбережения (L1).

22. Электротранспорт (Современный город)

- **Базовый класс:** `ElectricVehicle` (заряд АКБ, макс. пробег, текущая скорость).
- **Методы:** `charge()`, абстрактный `drive()`.
- **Наследники:**
 - `ElectricScooter`: метод `fold()` (сложить для переноски, блокирует запуск двигателя).
 - `Hoverboard`: метод `calibrateGyroscope()` (калибровка гироскопа для баланса).
 - `ElectricUnicycle`: метод `setPedalHardness()` (настройка жесткости педалей).
- **Логика:** При заряде < 10% скорость программно ограничивается (L1). Метод `drive()` невозможен, если устройство на зарядке (L3).

23. Измерительные приборы (Строительный профиль)

- **Базовый класс:** `MeasuringTool` (единицы измерения, точность/погрешность).
- **Методы:** `reset()`, абстрактный `measure()`.
- **Наследники:**
 - `LaserRangefinder`: метод `togglePointer()` (включить/выключить лазерную точку).
 - `DigitalLevel`: метод `checkAngle()` (проверка отклонения от горизонтали).
 - `ElectronicCaliper`: метод `switchScale()` (переключение мм/дюймы).
- **Логика:** Если линза лазера загрязнена, `measure()` выдает ошибку (L3). При простое более 5 минут — автоотключение (L1).

24. Устройства ввода (Гейминг и работа)

- **Базовый класс:** `InputDevice` (тип подключения: USB/BT, время отклика).
- **Методы:** `sync()`, абстрактный `sendSignal()`.
- **Наследники:**
 - `GamingMouse`: метод `setDPI(int value)` (изменение чувствительности сенсора).
 - `MechanicalKeyboard`: метод `setBacklight(enum Mode)` (смена режима подсветки).
 - `GraphicsTablet`: метод `setPressureSensitivity()` (настройка силы нажатия пера).
- **Логика:** Если соединение (`isConnected`) потеряно, `sendSignal()` инициирует повторную попытку связи (L1).

25. Кухонная техника для завтрака (Бытовой профиль)

- **Базовый класс:** `BreakfastMaker` (мощность, таймер, состояние нагрева).
- **Методы:** `preheat()`, абстрактный `cook()`.
- **Наследники:**
 - `WaffleMaker`: метод `setBrowningLevel()` (установка степени прожарки).
 - `SandwichPress`: метод `adjustClamp()` (регулировка прижима пластин).
 - `YogurtMaker`: метод `setFermentationTime()` (установка времени закваски).
- **Логика:** Метод `cook()` не запускается, пока не завершён `preheat()` (L2: сообщение пользователю).

26. Системы полива (Агро-технологии)

- **Базовый класс:** `IrrigationSystem` (давление воды, расход л/мин).
- **Методы:** `openValve()`, абстрактный `spray()`.
- **Наследники:**
 - `DripLine`: метод `checkClogging()` (проверка засорения отверстий).
 - `RotarySprinkler`: метод `setRotationArc(int arc)` (установка угла полива).
 - `Fogger`: метод `setMistDensity()` (настройка плотности тумана).

- **Логика:** При падении давления в магистрали ниже нормы, `spray()` переходит в режим ожидания (L1).

27. Аудио-устройства (Звукозапись)

- **Базовый класс:** `AudioDevice` (частотный диапазон, уровень усиления).
- **Методы:** `setGain()`, абстрактный `processAudio()`.
- **Наследники:**
 - `Microphone`: метод `setPolarPattern(enum Pattern)` (выбор направленности).
 - `AudioInterface`: метод `togglePhantomPower()` (включение фантомного питания +48V).
 - `StudioMonitor`: метод `adjustEQ(double hf, double lf)` (коррекция частот).
- **Логика:** Если включено фантомное питание, `processAudio()` проверяет совместимость устройства (L1/L2).

28. Умные носимые устройства (Wearables)

- **Базовый класс:** `Wearable` (версия ПО, пульс, шаги).
- **Методы:** `syncWithPhone()`, абстрактный `trackActivity()`.
- **Наследники:**
 - `FitnessTracker`: метод `startWorkout(enum Type)` (запуск тренировки).
 - `SmartWatch`: метод `showNotification(String msg)` (вывод уведомления).
 - `HeartRateMonitor`: метод `calibrateECG()` (калибровка датчика ЭКГ).
- **Логика:** Если ремешок не застегнут (датчик `onWrist == false`), измерения не производятся (L1).

29. Инструменты для мастерской (DIY-профиль)

- **Базовый класс:** `PowerTool` (обороты в минуту, температура двигателя).
- **Методы:** `start()`, абстрактный `execute()`.
- **Наследники:**
 - `ElectricDrill`: метод `setDirection(enum Dir)` (смена направления вращения).
 - `Jigsaw`: метод `setBladeAngle()` (установка угла наклона пилки).
 - `OrbitalSander`: метод `attachSandpaper(int grit)` (установка зернистости бумаги).
- **Логика:** При перегреве двигателя `execute()` блокируется до остывания (L1: принудительная пауза).

30. Климатическая техника (Промышленный профиль)

- **Базовый класс:** `IndustrialClimate` (объем помещения, заданная влажность).
- **Методы:** `analyzeAir()`, абстрактный `maintain()`.
- **Наследники:**
 - `Dehumidifier`: метод `drainCondensate()` (слив конденсата из бака).
 - `HeatCurtain`: метод `setAirVelocity()` (настройка скорости воздушного потока).
 - `AirPurifier`: метод `checkFilterStatus()` (проверка загрязнения HEPA-фильтра).
- **Логика:** Если фильтр загрязнен на 100%, `maintain()` снижает мощность и выводит сервисное сообщение (L1/L2).